CMS Advanced Transformer Model :: Complete User Workflow & Process Documentation

Version: 9.0 (Advanced CMS Integration) **Date:** March 2026
Purpose: End-to-End Process Guide for Users

Table of Contents

1. User Goals & Objectives
2. Complete Process Flow
3. Step-by-Step Workflow
4. Deep Learning Model Integration Explained
5. Output Guide
6. Quick Reference

---

1. User Goals & Objectives

What is the User Trying to Achieve?

As a researcher or engineer using the Chemical Modification System (CMS), your primary goal is:

> **Design modifications for an siRNA sequence so it survives in the body (Stability), evades the immune system (Safety), and successfully silences the target (Efficacy) without needing years of manual lab testing.**

The Three Pillars of a Good Modified siRNA

For an siRNA sequence to be successfully deployed, we apply Chemical Armor. This armor must balance **THREE conflicting requirements**:

```
┌──────────────────────────────────────────────────────────────────────────────
│                THE THREE PILLARS OF MODIFIED siRNA DESIGN
├──────────────────────────────────────────────────────────────────────────────
│
│    ┌──────────────────────┐
│    │      STABILITY       │ ← Will it survive blood enzymes?
│    │   (Half-Life)        │   Added armor increases this!
│    └──────────────────────┘
│            │
│            ▼
│    ┌──────────────────────┐
│    │    IMMUNOGENICITY    │ ← Will the immune system attack it?
```

```
│    │        (Safety)        │   Correct armor hides it!
│    └──────────────────────┘
│             │
│             ▼
│    ┌──────────────────────┐
│    │       EFFICACY       │ ← Can it still bind to the target?
│    │     (AGO2 Binding)   │   Too much armor makes it too stiff to work
│    └──────────────────────┘
│
│   ALL THREE must be perfectly balanced by the ML Model!
│
└──────────────────────────────────────────────────────────────────────────
```

---

2. Complete Process Flow

The Complete User Journey

```
┌──────────────────────────────────────────────────────────────────────────
│                          CMS MODEL COMPLETE WORKFLOW
└──────────────────────────────────────────────────────────────────────────

    ┌──────────────────────────────────────────────────────────────────────
    │                          STEP 1: USER INPUT
    │   ┌────────────────────┐          ┌────────────────────┐
    │   │ TARGET SEQUENCE    │          │ ARMOR BLUEPRINT    │
    │   │ (The siRNA)        │          │ (Modifications)    │
    │   │                    │          │                    │
    │   │ 21-Nucleotide str: │          │ Type: OME          │
    │   │ AUGCAUGCAUGCA...   │          │ Positions: 2, 5, 8 │
    │   └────────────────────┘          └────────────────────┘
    │           │                                  │
    └───────────┼──────────────────────────────────┼──────────────────────
                │                                  │
                ▼                                  ▼
    ┌──────────────────────────────────────────────────────────────────────
    │                    STEP 2: FEATURE ENGINEERING (ETL)
    │
    │   The raw string and armor positions are converted into 179 math
    │
    │    ┌──────────────────────────────────────────────────────────────
    │    │  Feature 1-50: Sequence Counts (G/C vs A/U ratios)
    │    │  Feature 51-100: Positional Penalties (How close is the armor
    │    │  Feature 101-125: Thermodynamics (Gibbs Free Energy calculati
    │    │  Feature 126-179: Chemical Constraints (Rigidity mapping)
    │    └──────────────────────────────────────────────────────────────
    └──────────────────────────────────────────────────────────────────────
                │
                ▼
    ┌──────────────────────────────────────────────────────────────────────
    │                    STEP 3: DEEP AI INFERENCE (GPU)
```

```
│
│   The 179-feature tensor passes into the Pytorch Transformer.
│
│    ┌──────────────────────────────────────────────────────────────────────
│    │                 4-LAYER TRANSFORMER ATTENTION
│    │
│    │  Layer 1: Self-Attention (How does Position 2 affect Position
│    │  Layer 2: Thermodynamic Weighting
│    │  Layer 3: Positional Embedding Correlation
│    │  Layer 4: Output Projection Branching
│    │                                                                    ▼
│    │   ┌──────────────────────────────────────────────────────────────────
│    │   │  Splits into 7 Independent Output Heads
│    │   └──────────────────────────────────────────────────────────────────
│    └──────────────────────────────────────────────────────────────────────
│
└──────────────────────────────────────────────────────────────────────────
            │
            ▼
┌──────────────────────────────────────────────────────────────────────────
│                    STEP 4: MULTI-TASK PREDICTION ROUTING
│
│    ┌──────────────────────────────────────────────────────────────────────
│    │                   7-HEAD OUTPUT GENERATION
│    │
│    │   1. Overall Therapeutic Index (0-100 float)
│    │   2. Efficacy Category (High/Medium/Low)
│    │   3. Half-life Prediction (Hours)
│    │   4. AGO2 Enzyme Accessibility Percentage
│    │   5. Immune Suppression Score (%)
│    │   6. RNase H Resistance Profile
│    │   7. Physical Stability (Nuclease Resistance)
│    └──────────────────────────────────────────────────────────────────────
└──────────────────────────────────────────────────────────────────────────
```

---

3. Step-by-Step Workflow

Phase 1: Submitting Data to the Application

You interact with the Flask Web Service (or via Python scripts) targeting the `/predict` endpoint.

- **You provide:** `sequence` (string), `modification_type` (like OME or FLUORO), and an array of `positions`.
- **The System:** Automatically translates aliases (converts `2_ome` into `OME`) using the Enum defined in `data_structures.py`.

Phase 2: Feature Extraction (The Math)

A Deep Learning model cannot read letters. The `FeatureExtractor` class translates the

Biological Rules into vectors.

- It parses the thermodynamics—calculating if the ends of the biological string are loose enough to easily unzip in the human body.
- It calculates the precise impact of the selected armor: Fluoro adds rigidity, Phosphorothioate adds chemical resistance.
- **Output:** A strict 179-element tensor.

Phase 3: AI Inference (The Brain)

- The PyTorch tensors are sent to the GPU (`device=cuda`).
- The 4 layers of Multi-Head Self-Attention analyze the string. Instead of reading left-to-right, the model simultaneously looks at all points against all other points.
- It calculates: *If we put armor on Position 2 AND Position 18... do they fold and hit each other in 3D space?*
- The final layer branches out to predict seven different biological outcomes simultaneously (Multi-Task Learning).

Phase 4: Delivery

- The Float32 numerical tensors are returned from the GPU, parsed back into simple Python floats, and formatted into clean JSON.

---

4. Deep Learning Model Integration Explained

Why Multi-Task Learning?

In older iterations of our tools, we predicted Efficacy and Stability completely separately. We discovered that **they are intrinsically linked**. By using a shared neural backbone with 7 different heads, the model learns the hidden representations connecting stability to toxicity.

Why Transfomers?

Older sequence-analyzers used RNNs (Recurrent Neural Networks), which read strings left-to-right. Biology operates in 3D physical folding. Position 1 can physically touch Position 21. **Transformers with Self-Attention** can instantly calculate the relationship between every combination of nucleotides regardless of distance.

Overcoming Biological Failure Rates (Focal Loss)

Highly effective siRNAs are incredibly rare. Standard AI models become "lazy" and just predict "Failure" 90% of the time, looking artificially accurate. We instituted **Focal Loss** ($\alpha = 0.25, \gamma = 2.0$), a mathematical formula that exponentially penalizes the network for incorrectly predicting the rare highly-successful sequences.

---

5. Output Guide

When you hit the system, you get a JSON response. Here is how to interpret the results:

Output Property	What it Means	Good vs Bad
`therapeutic_index`	The ultimate master score combining all targets.	>80 is Excellent
`efficacy_category`	Categorical bracket (0 to 3 scale)	3 is High Efficacy
`half_life`	How long the strand survives floating in human blood	>5 hours is great
`immune_suppression`	How well the sequence hides from the immune system	>80% means no cytokine storm
`ago2_binding`	How easily the cellular machinery can grab and use the siRNA	>75% means high activity
`rnase_h_resistance`	Resistance to specific RNase H clearing enzymes	>85% is ideal

---

6. Quick Reference

Supported Enhancements (Armor Types):

- `OME` (2'-O-Methyl): High immune suppression, mild stability boost.
- `FLUORO` (2'-Fluoro): Extreme stability boost, high binding affinity.
- `PS` (Phosphorothioate): Massive enzyme resistance, mild toxicity increase.

System Dependencies:

- PyTorch 2.0+ (CUDA Accelerated GPU capabilities)
- Flask 3.0+ (Web Server)
- Numpy (Float32 Array engineering)

Primary File Mapping:

1. Server parsing: `app.py`
2. Math conversion: `src/features.py`
3. Engine Execution: `src/model.py`
4. Scientific Rules: `src/data_structures.py`

Generated for the Advanced CMS Platform.